

FUNDAMENTOS DE ARQUITECTURA DE SOFTWARE

1. **INTRODUCCIÓN Y OBJETIVOS** LA ARQUITECTURA DE SOFTWARE SE DEFINE (BAJO EL ESTÁNDAR ISO/IEC/IEEE 42010) COMO LA ESTRUCTURA FUNDAMENTAL DE UN SISTEMA, COMPUESTA POR SUS COMPONENTES, LAS RELACIONES ENTRE ELLOS Y LOS PRINCIPIOS QUE GUÍAN SU DISEÑO.

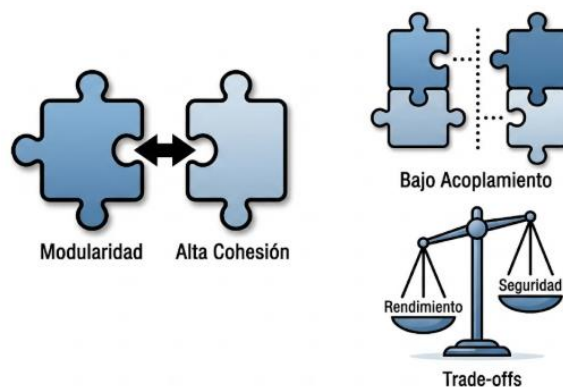
- **OBJETIVO PRINCIPAL:**

Establecer una base sólida que asegure el cumplimiento de requisitos funcionales (lo que hace el sistema) y no funcionales (rendimiento, seguridad, escalabilidad).

- **IMPORTANCIA:**

Reduce riesgos técnicos, mejora la calidad (usabilidad, mantenibilidad) y facilita la comunicación entre los interesados (stakeholders).

2. **PRINCIPIOS DE DISEÑO ARQUITECTÓNICO PARA QUE UNA ARQUITECTURA SEA ROBUSTA, DEBE SEGUIR CIERTOS PRINCIPIOS:**



- **MODULARIDAD:**

Dividir el sistema en módulos independientes.

- **COHESIÓN Y ACOPLAMIENTO:**

Se busca una alta cohesión (que cada módulo tenga una tarea clara) y un bajo acoplamiento (mínima dependencia entre módulos).

- **ESCALABILIDAD:**

Capacidad de adaptarse al crecimiento de usuarios o datos.

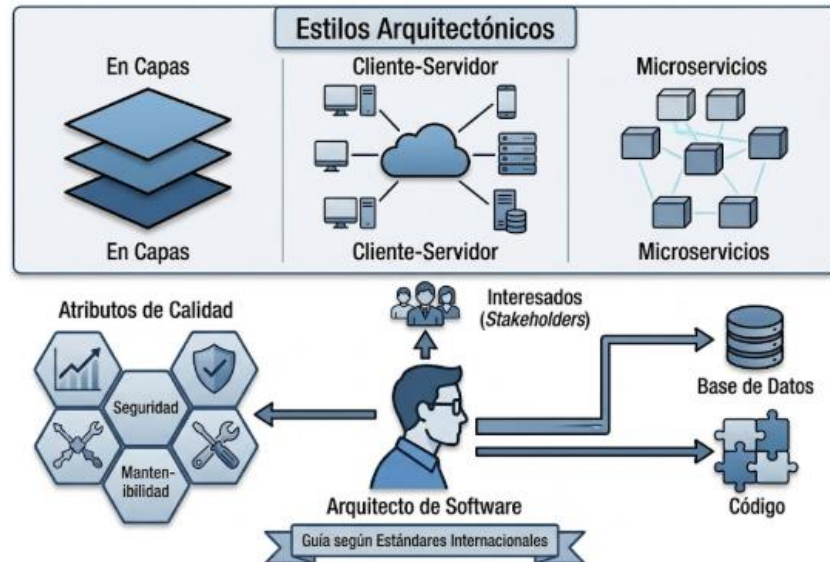
- **MANTENIBILIDAD Y REUTILIZACIÓN:**

Facilidad para corregir errores y usar componentes en otros proyectos.

- **BALANCE DE TRADE-OFFS:**

Las decisiones implican equilibrio; por ejemplo, aumentar la seguridad podría reducir un poco el rendimiento.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS LOS ESTILOS DEFINEN CÓMO SE ORGANIZA EL SISTEMA A NIVEL GENERAL: ESTILOS TRADICIONALES:



4. ARQUITECTURA EN CAPAS:

Divide el sistema en niveles jerárquicos (Presentación, Lógica, Datos).

5. CLIENTE-SERVIDOR:

El cliente solicita recursos y el servidor los provee.

6. SOA (ORIENTADA A SERVICIOS):

Servicios independientes que interactúan mediante protocolos estándar.

7. MICROSERVICIOS:

Evolución de SOA; servicios pequeños, autónomos y desplegados de forma independiente. Patrones Frecuentes:

- **MVC (MODELO-VISTA-CONTROLADOR):**

Separa la lógica de negocio de la interfaz.

- **MVVM (MODELO-VISTA-VISTAMODELO):**

Ideal para aplicaciones con data binding (móviles/web modernas).

- **REPOSITORY PATTERN:**

Abstrae el acceso a los datos.

- **PIPES AND FILTERS:**

Los datos fluyen a través de una serie de filtros de procesamiento.

4. MODELADO Y DOCUMENTACIÓN ES VITAL REPRESENTAR LA ARQUITECTURA DE FORMA VISUAL Y COMPRENSIBLE UTILIZANDO:

- **UML (UNIFIED MODELING LANGUAGE):**

Incluye diagramas de Casos de Uso (funcionalidad), Componentes (estructura lógica), Despliegue (infraestructura física) y Secuencia (interacción temporal).

- **MODELO C4:**

Organiza la arquitectura en 4 niveles de detalle progresivo:

1. **Contexto:** El sistema y su entorno.
2. **Contenedores:** Aplicaciones, bases de datos, etc.
3. **Componentes:** Organización interna de cada contenedor.
4. **Código:** Detalles de implementación (clases/interfaces).

5. ESTÁNDARES Y ROLES

- **ESTÁNDARES:**

Se mencionan ISO/IEC/IEEE 42010 (descripción de arquitecturas), TOGAF (marcos empresariales) y SEI/ATAM (evaluación de atributos de calidad).

- **EL ARQUITECTO DE SOFTWARE:**

Es el responsable de definir la estructura global, seleccionar patrones, garantizar requisitos no funcionales y guiar al equipo de desarrollo.

- **RAZONAMIENTO ARQUITECTÓNICO:**

Es la justificación de por qué se tomaron ciertas decisiones, documentando siempre los beneficios y las desventajas (trade-offs).